# EXPERIMENT - 14

**AIM :**

To implement the Alpha-Beta Pruning algorithm to reduce unnecessary searches in a game tree.

**ALGORITHM:**

1. Start the program.
2. Initialize alpha to negative infinity and beta to positive infinity.
3. Evaluate the game tree recursively.
4. At the MAX level, update alpha with the maximum value.
5. At the MIN level, update beta with the minimum value.
6. Prune a branch when alpha is greater than or equal to beta.
7. Display the best value and stop.

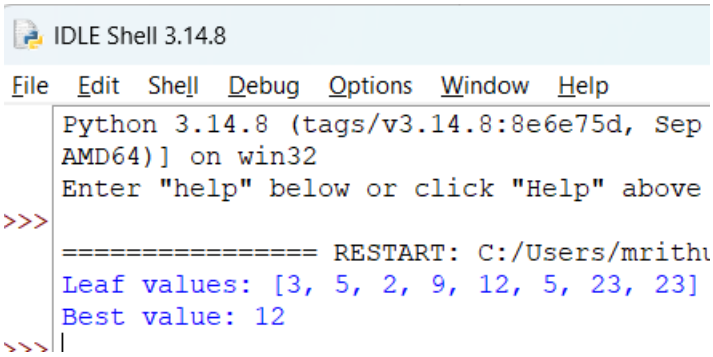
**PROGRAM :**

*untitled (3.14.8)*

File Edit Format Run Options Window Help

```
def alphabeta(depth, node, is_max, values,
              alpha, beta, max_depth):

    if depth == max_depth:
        return values[node]

    if is_max:
        best = float('-inf')

        for child in [node * 2, node * 2 + 1]:
            value = alphabeta(
                depth + 1, child, False, values
                alpha, beta, max_depth
            )
            best = max(best, value)
            alpha = max(alpha, best)

            if alpha >= beta:
                break

        return best

    else:
        best = float('inf')

        for child in [node * 2, node * 2 + 1]:
            value = alphabeta(
                depth + 1, child, True, values,
                alpha, beta, max_depth
            )
            best = min(best, value)
            beta = min(beta, best)

            if alpha >= beta:
                break

        return best

values = [3, 5, 2, 9, 12, 5, 23, 23]

result = alphabeta(
    0, 0, True, values,
    float('-inf'), float('inf'), 3
)

print("Leaf values:", values)
print("Best value:", result)
```

**OUTPUT:**




**RESULT:**

Thus, the Alpha-Beta Pruning algorithm was implemented successfully to find the optimal value while avoiding unnecessary node evaluations.